

1. 公开 API

1.1. 使用说明

1.1.1. 公共请求说明

API 接口请求参数包括公共参数和自定义业务参数两部分。公共请求参数是调用每个 API 时都需要携带的请求参数，包括服务命名空间、接口名称、版本信息。自定义业务参数由各接口定义，根据调用方法不同，需要将参数携带至请求路径或者请求体中。API 接口公共参数调用如下所示：

https(http)://xxxx.com/{namespace}?action=xxxx&version=1

参数说明：

序号	参数	类型	是否必选	描述
1	namespace	string	是	API 接口类别，目前支持 common 设备管理类
2	action	string	是	API 接口名称
3	version	string	是	API 版本号，目前所有 API 接口版本均为 1

1.1.2. 公共响应说明

成功响应：

```
{
  "requestId": "8906582E6722409AA6C40E7863B733A5",
  "success": true,
  "data": {
    status: 1
  }
}
```

失败响应：

```
{
  "requestId": "8906582E6722409AA6C40E7863B733A5",
  "code": "iot.application.deviceNotFound",
  "msg": "device does not exist",
  "success": false
}
```

参数说明:

序号	参数	类型	描述
1	requestId	string	请求 ID，调用 API 时由平台生成唯一请求标识
2	code	string	调用失败时，返回的错误码
3	msg	string	调用失败时，返回的错误信息
4	success	boolean	接口是否调用成功
5	data	object	调用成功时，返回的业务数据（接口无业务数据返回，值为 null）

1.2. 安全鉴权

平台需要对 API 调用方进行资源权限校验，使用 API 时，需要在请求 Header 中携带统一的安全鉴权信息。

(1)安全鉴权机制

安全鉴权 authorization 由多个参数构成，每个参数均采用 key = value 的形式表示，并用&作为分隔符：

authorization:

version=2020-05-29&res=userid%2F38055&et=1623982416&method=sha1&sign=SO4GcvafYIjtAMHJthkGPevbNwE%3D

参数说明:

序号	参数	类型	说明	示例
1	version	string	签名算法版本	目前仅支持 2020-05-29
2	res	string	访问资源信息	支持主用户访问方式： 1) 主用户访问 res 为: userid/{userid}, userid 为平台用户 id, 参数在个人「账号信息」中查看
3	et	string	访问过期时间	10 位时间戳, 1537255523 表示: 北京时间 2018-09-18 15:25:23
4	method	string	签名方法	目前支持 md5、sha1、sha256
5	sign	string	签名结果字符串	version、res、et、method 参数计算生成

其中 sign 的生成算法为:

```
sign = base64(hmac_<method>(base64decode(accessKey), utf-8(StringForSignature)))
```

laccessKey 为平台分配的访问密钥（用户访问权限页面查看），如果访问资源以主用户形式访问，使用主用户的 accessKey，如果访问资源以项目群组方式访问，使用群组的 accessKey，且只能对群组内设备进行操作。

laccessKey 参与计算前应先进进行 base64decode 操作

l 用于计算签名的字符串 StringForSignature 按照 et、method、resource、version 的顺序，以"\n"作为分隔符进行排列，如下所示：

```
StringForSignature = et + "\n" + method + "\n" + res + "\n" + version
```

(2)参数编码

authorization 中 key=value 的形式的 value 部分需要经过 URL 编码，需要进行编码的特殊符号如下：

序号	符号	编码
1	+	%2B
2	空格	%20
3	/	%2F
4	?	%3F
5	%	%25
6	#	%23
7	&	%26
8	=	%3D

(3)authorization 生成示例

Ønodejs 代码示例

```
'use strict';
```

```
const crypto = require('crypto');
```

```
/**
```

```
 * authorization 生成函数
```

```
 *
```

```
 * @param {String} method          - hash method
```

```
 * @param {String} res             - resource
```

```
 * @param {String} accessKey       - access key
```

```
 * @param {Number} et              - effective time
```

```
 * @return {String} - authorization
```

```
 */
```

```
function generateAuthorization(method, res, accessKey, et) {
```

```
    const version = '2020-05-29';
```

```
    const et = Math.ceil((Date.now() + et) / 1000); // token 有效时间
```

```
    const base64Key = Buffer.from(accessKey, 'base64'); // accessKey base64 编码
```

```

    const StringForSignature = et + '\n' + method + '\n' + res + '\n' + version;

    const sign = encodeURIComponent(crypto.createHmac(method,
base64Key).update(StringForSignature).digest('base64'));

    const encodeRes = encodeURIComponent(res);

    return
`version=${version}&res=${encodeRes}&et=${et}&method=${method}&sign=${sign}`;
}

```

```

const method = 'sha1';
const accessKey =
'mjgvkTCYTBf6DguxMmm+aV9EkDp2CYfL5jzRTph5Th6KhU8gqZz/cBivPTA7tfY5';
const res = 'userid/130037';
const et = 365 * 24 * 3600 * 1000; // 有效时间 - 365 天
const authorization = generateAuthorization(method, res, accessKey, et);

```

Øpython 代码示例

```

import base64

import hmac

import time

from urllib.parse import quote

def token(user_id,access_key):

    version = '2020-05-29'

    res = 'userid/%s' % user_id

    # 用户自定义 token 过期时间
    et = str(int(time.time()) + 3600)

```

```

# 签名方法, 支持 md5、sha1、sha256
method = 'sha1'

# 对 access_key 进行 decode
key = base64.b64decode(access_key)

# 计算 sign
org = et + '\n' + method + '\n' + res + '\n' + version
sign_b = hmac.new(key=key, msg=org.encode(), digestmod=method)
sign = base64.b64encode(sign_b.digest()).decode()

# value 部分进行 url 编码, method/res/version 值较为简单无需编码
sign = quote(sign, safe='')
res = quote(res, safe='')

# token 参数拼接
token = 'version=%s&res=%s&et=%s&method=%s&sign=%s' % (version, res,
et, method, sign)

return token

if __name__ == '__main__':
    user_id = '37715'
    access_key =
'mjgvkTCYTBF6DguxMmm+aV9EkDp2CYfL5jzRTph5Th6KhU8gqZz/cBivPTA7tfY5'

    print(token(user_id,access_key))

```

ØJava 代码示例

```

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.io.UnsupportedEncodingException;

```

```

import java.net.URLEncoder;

import java.security.InvalidKeyException;

import java.security.NoSuchAlgorithmException;

import java.util.Base64;


public class Token {


    public static String assembleToken(String version, String resourceName,
String expirationTime, String signatureMethod, String accessKey)
        throws UnsupportedOperationException,
NoSuchAlgorithmException, InvalidKeyException {

        StringBuilder sb = new StringBuilder();

        String res = URLEncoder.encode(resourceName, "UTF-8");

        String sig = URLEncoder.encode(generatorSignature(version,
resourceName, expirationTime, accessKey, signatureMethod), "UTF-8");

        sb.append("version=")

            .append(version)

            .append("&res=")

            .append(res)

            .append("&et=")

            .append(expirationTime)

            .append("&method=")

            .append(signatureMethod)

            .append("&sign=")

            .append(sig);

        return sb.toString();
    }


    public static String generatorSignature(String version, String resourceName,
String expirationTime, String accessKey, String signatureMethod)

```

```

        throws NoSuchAlgorithmException, InvalidKeyException {
        String encryptText = expirationTime + "\n" + signatureMethod + "\n" +
resourceName + "\n" + version;
        String signature;
        byte[] bytes = HmacEncrypt(encryptText, accessKey, signatureMethod);
        signature = Base64.getEncoder().encodeToString(bytes);
        return signature;
    }

```

```

public static byte[] HmacEncrypt(String data, String key, String
signatureMethod)

```

```

        throws NoSuchAlgorithmException, InvalidKeyException {
//根据给定的字节数组构造一个密钥,第二参数指定一个密钥算法
的名称

```

```

        SecretKeySpec signinKey = null;
        signinKey = new SecretKeySpec(Base64.getDecoder().decode(key),
            "Hmac" + signatureMethod.toUpperCase());

```

```

//生成一个指定 Mac 算法 的 Mac 对象

```

```

        Mac mac = null;
        mac = Mac.getInstance("Hmac" + signatureMethod.toUpperCase());

```

```

//用给定密钥初始化 Mac 对象

```

```

        mac.init(signinKey);

```

```

//完成 Mac 操作

```

```

        return mac.doFinal(data.getBytes());
    }

```

```

public enum SignatureMethod {

```



```

        SHA1, MD5, SHA256;
    }

    public static void main(String[] args) throws UnsupportedEncodingException,
NoSuchAlgorithmException, InvalidKeyException {
        String version = "2020-05-29";
        String resourceName = "userid/12321";
        String expirationTime = System.currentTimeMillis() / 1000 + 100 * 24 *
60 * 60 + "";
        String signatureMethod = SignatureMethod.SHA1.name().toLowerCase();
        String accessKey =
"KuF3NT/jUBJ62LNBB/A8XZA9CqS3Cu79B/ABmfA1UCw=";
        String token = assembleToken(version, resourceName, expirationTime,
signatureMethod, accessKey);
        System.out.println("Authorization:" + token);
    }
}

```

Øgo 代码示例

```

func (s *SignService) GenerateSign(params_map map[string]string, access_key string)
(string, error){
    signature_str := params_map["et"] + "\n" + params_map["method"] + "\n" +
params_map["res"] + "\n" + params_map["version"]
    hmac_str := ""
    switch strings.ToLower(params_map["method"]) {
        case "sha1":
            hmac_str = tools.HmacSha1(signature_str,
tools.Base64Decode(access_key))
        case "sha256":
            hmac_str = tools.HmacSha256(signature_str,
tools.Base64Decode(access_key))
        case "md5":

```

```

        hmac_str = tools.HmacMd5(signature_str,
tools.Base64Decode(access_key))

        default:

            return "",errors.New("签名参数错误! ")

    }

    sign :=tools.Base64Encode(hmac_str)

    return sign,nil
}

func Base64Encode(message string) string {
    return base64.StdEncoding.EncodeToString([]byte(message))
}

func Base64Decode(message string) string {
    decode_str, err := base64.StdEncoding.DecodeString(message)
    if err != nil {
        return ""
    }
    return string(decode_str)
}

func HmacSha256(data string, secret string) string {
    h := hmac.New(sha256.New, []byte(secret))
    h.Write([]byte(data))
    return string(h.Sum(nil))
    //return hex.EncodeToString(h.Sum(nil))
}

func HmacMd5(data string, secret string) string {

```

```

    h := hmac.New(md5.New, []byte(secret))
    h.Write([]byte(data))
    return string(h.Sum(nil))
    //return hex.EncodeToString(h.Sum(nil))
}

func HmacSha1(data string, secret string) string {
    h := hmac.New(sha1.New, []byte(secret))
    h.Write([]byte(data))
    return string(h.Sum(nil))
    //return hex.EncodeToString(h.Sum(nil)) //不需以十六进制字符串输出
}

```

Øphp 代码示例

```

class SafetyAuth
{
    private  $_version = '2020-05-29';//版本
    private  $_method = 'sha1';//加密方法，还可以用 md5、sha256
    private  $_access_key;//访问密钥
    private  $_res ;//资源参数
    private  $_et;//到期时间戳
    private  $_expiration;//有效期，有效时间

    public function __construct($access_key, $expiration,$res){
        $this->_expiration = $expiration;
        $this->_access_key = $access_key;
        $this->_res = $res;
    }

    //生成 sign
    private function _makeSign() {

```

```

        date_default_timezone_set('PRC');

        $this->_et =time() + $this->_expiration;

        $string_for_signature = $this->_et . "\n" . $this->_method . "\n" .
$this->_res . "\n" . $this->_version;

        $b64_decode_acckey = base64_decode($this->_access_key);

        $hmac_key = hash_hmac($this->_method,
$this->_strToUtf8($string_for_signature), $b64_decode_acckey, true);

        $sign = base64_encode($hmac_key);

        return $sign;

    }

    private function _strToUtf8($str){

        $encode = mb_detect_encoding($str,
array("ASCII",'UTF-8',"GB2312","GBK",'BIG5'));

        if($encode == 'UTF-8'){

            return $str;

        }else{

            return mb_convert_encoding($str, 'UTF-8', $encode);

        }

    }

    //生成 token

    public function makeToken() {

        $sign = $this->_makeSign();

        $token =

sprintf("version=2020-05-29&res=%s&et=%s&method=sha1&sign=%s",
urlencode($this->_res), $this->_et, urlencode($sign));

        return $token;

    }

}

```

1.3. 错误码

本文档列举 API 调用失败时，返回的错误码。出现缺少请求参数、不合法的请求参数等错误时，请参见具体 API 描述进行修改。

资源权限相关错误码：

错误码	描述
authPermissionDeny	鉴权失败
resourePermissionDeny	无资源访问权限
parameterRequired	缺少请求参数
parameterMissing	缺少请求参数
invalidParameter	不合法的请求参数

产品相关错误码：

错误码	描述
productNotFound	产品不存在
productHasNoDevice	产品未创建设备

设备相关错误码：

错误码	描述
deviceNotFound	设备不存在
setPropertyFailed	设备属性设置失败
setDesiredPeropertyFailed	设备属性期望设置失败
queryDesiredProperyFailed	设备属性期望查询失败
getTmProperties	设备属性获取失败
callTmService	设备服务调用失败
deleteDesiredPeropertyFailed	设备属性期望删除失败

queryCurrentData	设备最新数据查询失败
queryPropertyHistoryData	设备属性历史数据查询失败
queryEventHistoryData	设备事件历史数据查询失败
queryOperationLog	设备操作记录查询失败

1.4. API 列表

1.4.1. 产品管理

1.4.1.1. 产品详情

方法	GET			
路径 URI	/common?action=ProductDetail&version=1&product_id=10132			
请求头				
URL 参 数	product_id	string	必填	产品 id
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.device_number	number	设备数量	
	data.product_id	string	产品 ID	
	<u>data.name</u>	string	产品名称	

	data.desc	string	产品描述
	data.network	string	联网方式
	data.category	string	产品所属分类 ID
	data.category_name	string	产品所属分类名称
	data.protocol	number	协议类型 1-泛协议 2-MQTT 3-CoAP
	data.create_time	string	创建时间
	data.ind_title	string	产品行业名称
	data.prod_ind	string	产品行业编码
	data.prod_chain	array	产品行业编码层级关系
	data.uid	string	产品用户 ID
	data.sec_key	string	产品权限 key
	data.template_id	string	模板 id
	data.template_info	array	模板信息
请求示例	GET /common?action=ProductDetail&version=1&product_id=10132		
响应示例	<pre>{ "data": { "device_number":0, "protocol":2, "created_time":"2021-12-06T08:28:56.762Z", "product_id":"wjrJjSRtsG", "network":"1", "category":"138", "ind_title":"智慧城市", "prod_ind":"1",</pre>		

	<pre>"uid":37782, "sec_key":"vC3eX2gr8mapsNZ1zYVmtsFRMYaX953GbJiKlcAQTp4=", "desc":"", "name":"物模型", "template_id": "6406f2ee9641f20035c53b12", "template_info": [] }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	--

1.4.1.2. 产品列表

方法	GET			
路径 URI	/common?action=ProductList&version=1			
请求头				
URL 参 数	name	string	可选	产品名称
	manufacturer	string	可选	厂商名称
	protocol	string	可选	接入协议, 可选['1', '2', '3', '4', '5', '6', '7', '8', '9', '0']
	industry	string	可选	行业类型编码
	offset	string	可选	请求起始位置, 默认 0
	limit	string	可选	每次请求记录数, 默认 10
请求体 参数	无			
响应参 数	code	string	调用失败时, 返回的错误码	
	msg	string	调用失败时, 返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	

	success	boolean	接口是否调用成功
	data	—	调用成功时, 返回业务数据
	data.list	array	产品信息集合, 如下的l表示 list 数组的单个对象标识
	l. total	number	设备数量
	l. desc	string	描述
	l. protocol	number	协议 1-泛协议 2-MQTT 3-CoAP
	l. created_time	string	创建时间
	l. product_id	string	产品 ID
	l. network	string	联网模式 1-其他 2-蜂窝 3-wifi 4-以太网
	l. ind_title	string	产品行业名称
	l. prod_ind	string	产品行业编码
	l. uid	string	产品用户 ID
	l. sec_key	string	产品权限 KEY
	l. name	string	产品名称
	l. manufacturer	string	厂商名称
	l.template_id	string	模板 id
	l.prod_chain	array	产品行业编码层级关系
	l. category	array	分类 ID
	l.category_name	string	分类名称
	data.meta	object	分页信息
	data.meta.limit	number	每次请求记录数
	data.meta.offset	number	请求记录起始位置

	data.meta.total	number	条数
请求示例	GET /common?action=ProductList&version=1&product_id=wA10WBynvt		
响应示例	<pre> { "data": { "list": [{ "total": 2, "protocol": 4, "created_time": "2021-08-31T08:00:12.846Z", "product_id": "wA10WBynvt", "network": "3", "category": ["66006c672de43d02590b098b"], "ind_title": "智慧城市", "prod_ind": "1", "prod_chain": ["3", "9"], "uid": 5, "template_id": "6687c206537561748c69f470", "sec_key": "RJKrcCGT22DYGSQ4KVXslu/Jnh3bezHzvhqCTFMk05Q=", "desc": "", "name": "qwe123", "manufacturer": "S1234567890", "category_name": "智能后视镜" }], "meta": { "total": 1, "limit": 10, "offset": 0 } } } </pre>		

	<pre> }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>

1.4.1.3. 产品设备列表

方法	GET			
路径 URI	/common?action=DeviceList&version=1			
请求头				
URL 参数	name	string	可选	设备名称
	product_id	string	可选	产品 ID
	status	string	可选	设备状态, 可选['1'未激活, '2'在线, '3'离线]
	offset	string	可选	请求起始位置, 默认 0
	limit	string	可选	每次请求记录数, 默认 10
请求体参数	无			
响应参数	code	string	调用失败时, 返回的错误码	
	msg	string	调用失败时, 返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时, 返回业务数据	
	data.list	array	产品信息集合, 如下的 l 表示 list 数组的单个对象标识	
	l. product_id	string	产品 ID	
	l. name	string	设备名称	

	l. node_type	number	节点类型 1: 直连设备, 2: 网关设备, 3: 网关子设备
	l. status	number	设备状态 1: 未激活, 2: 在线, 3: 离线 【默认为未激活】
	l. last_time	string	设备最后一次在线时间
	l. created_time	string	创建时间
	l. from	string	设备来源(1:自主创建, 2:他人转移)
	l. lon	string	经度
	l. lat	string	纬度
	l. idid	string	设备 ID
	l. product_name	string	产品名称
	l.scene	array	场景 id
	data.meta	object	分页信息
	data.meta.limit	number	每次请求记录数
	data.meta.offset	number	请求记录起始位置
	data.meta.total	number	条数
请求示例	GET /common?action=DeviceList&version=1		
响应示例	<pre>{ "data": { "list": [{ "pid": "7ubgKi1vhm", "name": "400A002090011001", "ct": "2021-11-30T08:22:53.519Z",</pre>		

	<pre>"node_type":1, "status":3, "last_time":"2021-12-10T10:19:33.327Z", "from":1, "lon":""," "lat":""," "idid":"10015446", "product_id":"7ubgKi1vhm", "created_time":"2021-11-30T08:22:53.519Z", "id":10015446, "product_name":"水电费", "scene":["a25087f46df040ef0acfd115"] }], "meta":{ "total":1, "limit":10, "offset":0 } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	--

1.4.1.4. 产品设备数量统计

方法	GET			
路径 URI	/common?action=ProductDeviceStatistics&version=1			
请求头				
URL 参数	product_id	string	必填	产品 id

请求体参数	无		
响应参数	code	string	调用失败时，返回的错误码
	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据
	data.unactive	number	未激活设备数
	data.online	number	在线设备数
	data.offline	number	离线设备数
	data.total	number	总的设备数
请求示例	GET /common?action=ProductDeviceStatistics&version=1&product_id=7ubgKilvhm		
响应示例	<pre>{ "data": { "unactive":6, // 未激活数 "online":0, // 在线数 "offline":8, // 离线数 "total":14 // 总数 } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.2. 设备管理

1.4.2.1. 设备创建

方法	POST			
路径 URI	/common?action=CreateDevice&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	imei	string	可选 (LwM2M 协议时必 填)	NB 设备 imei, 15 个数字组成的电子串号, 设备所属产品为 LwM2M 时, 为必填
	imsi	string	可选	NB 设备 imsi, 不超过 15 个的数字, 设备所属产品为 LwM2M 时, 为必填
	psk	string	可选	NB 设备所需属性, 若创建未填则平台随机生成, 8-16 位数字字母组合
	template_field	object	可选	如产品有选择设备档案模版, 根据所选模版构造的 json 对象
	auth_code	string	可选	NB 设备鉴权码, 1-16 位数字字母组合
	desc	string	可选	设备描述
	tag	object	可选	设备标签
	scene	string	必填	场景_id
	supplier	numbe	必填	供应商 id

		r		
	template_field	object	可选	设备档案数据
响应参数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	——		调用成功返回业务数据
	<u>data.name</u>	string		设备名称
	data.node_type	string		节点类型 1-直连设备
	data.desc	string		设备描述
	data.sec_key	string		设备密钥
	data.protocol	int		协议 1-泛协议 2-MQTT 3-CoAP
	data.created_time	string		创建时间
	data.imei	string		NB 设备 imei，15 个数字组成的电子串号
	data.imsi	string		NB 设备 imsi，不超过 15 个的数字
	data.auth_code	string		NB 设备鉴权码
	data.psk	string		NB 设备所需属性
	data.tag	object		设备标签
	data.template_field	object		设备档案数据
	data.template_id	string		模板 ID
请求示例	{ "product_id": "9MaNe52pNO",			

	<pre> "device_name": "no003", "imei": "366322456556584", "imsi": "366322456556584", "auth_code": "authcode", "psk": "authcode", "desc": "iot application", "tag": {"tagkey": "tagvalue"}, "template_field": {"key": "value"} } </pre>
响应示例	<pre> { "data": { "name": "no003", "node_type": 1, "desc": "iot application", "sec_key": "imQE5CGsIiT9ZMBMD/bSbqMnPIBwXXsYynQQsi/fimk=", "created_time": "2020-06-08T10:33:30.442Z", "protocol": 1, "tag": {"tagkey": "tagvalue"}, "template_field": {"key": "value"}, }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>

1.4.2.2. 批量创建设备

方法	POST
路径 URI	/common?action=BatchCreateDevices&version=1

请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	product_id	string	必填	产品 ID
	devices	array	必填	批量创建的设备信息集合, 一次最多创建 500 个设备。每个集合元素为 json 对象, 包括 name、desc、scene、supplier 和档案数据属性值, 例如[{ "name": "no001" , "scene": "660a1a925599c5007e1991cd" , "supplier": 5, "desc": "iot application" ,"template_field": {"key":"value"}}], 设备为 LwM2M 协议时, 增加 imei、imsi、auth_code、psk 属性, 如产品有选择设备档案模版增加 template_field 属性, 校验规则请参考设备创建接口
响应参 数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	— —		调用成功返回业务数据
	data.list	array		创建成功设备信息集合, 如下的 l 表示 list 数组的单个对象标识
	l.name	string		设备名称
	l.node_type	Int		设备类型 1-直连设备
	l.desc	string		设备描述
	l.sec_key	string		设备密钥

	l.protocol	int	协议类型 1-泛协议 2-MQTT 3-CoAP
	l.imei	string	imei
	l.imsi	string	imsi
	l.auth_code	string	auth_code
	l.psk	string	psk
	l.created_time	string	创建时间
	l.template_fiel d	object	设备档案数据
	l.template_id	string	模板 ID
请求示 例	<pre>{ "product_id": "9MaNe52pNO", "devices": [{ "name": "no001", "desc": "iot application", "scene": "660a1a925599c5007e1991cd", "supplier": 5, "template_field": {"key": "value"} }] }</pre>		
响应示 例	<pre>{ "data": { list: [{ "name": "no001", "node_type": 1,</pre>		

	<pre> "desc": "iot application", "sec_key": "imQE5CGsIiT9ZMBMD/bSbqMnPIBwXXsYynQQsi/fimk=", "created_time": "2020-06-08T10:33:30.442Z", "protocol": 1, "template_id": "6406f2ee9641f20035c53b12", "template_field": {"key": "value"} } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	--

1.4.2.3. 设备编辑

方法	POST			
路径 URI	/common?action=UpdateDevice&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	imsi	string	可选	NB 设备 imsi，不超过 15 个的数字，设备所属产品为 LwM2M 时，为必填
	psk	string	可选	NB 设备所需属性，若创建未填则平台随机生成，8-16 位数字字母组合

	auth_code	string	可选	NB 设备鉴权码, 1-16 位数字字母组合
	desc	string	可选	设备描述
	tag	object	可选	设备标签
	template_field	object	可选	设备档案数据
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	——		调用成功返回业务数据
	data.device_name	string		设备名称
	data.desc	string		设备描述
	data.tag	object		设备标签
	data.template_fiel d	object		设备档案数据
请求示例	{ "product_id": "9MaNe52pNO", "device_name": "no003", "desc": "iot application", "tag": {"tagkey": "tagvalue"}, "template_field": {"key": "value"} }			
响应示例	{ "data": { "device_name": "no003", "desc": "iot application", "tag": {"tagkey": "tagvalue"},			

	<pre>"template_field": {"key": "value"} }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	---

1.4.2.4. 设备删除

方法	POST			
路径URI	/common?action=DeleteDevice&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
请求示例	{ "product_id": "9MaNe52pNO", "device_name": "no003" }			
响应示例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }			

	}
--	---

1.4.2.5. 设备详情

方法	GET			
路径 URI	/common?action=QueryDeviceDetail&version=1&product_id=lsibd9 &device_name=no001			
请求头				
URL 参数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
请求体参数	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.device_name	string	设备名称	
	data.product_id	string	产品 ID	
	data.product_name	string	产品名称	
	data.desc	string	设备描述	
	data.status	int	设备状态 1-未激活 2-在线 3-离线	
	data.node_type	int	节点类型 1-直连设备	
	data.protocol	int	协议类型 1-泛协议 2-MQTT 3-CoAP	
	data.ip	string	设备连接 ip	

	data.create_time	string	创建时间
	data.last_time	string	最后一次在线时间
	data.active_time	string	激活时间
	data.sec_key	string	设备密钥
	data.template_id	string	所用的模板 id
	data.template_field	object	模板中添加的数据
请求示例	GET /common?action=QueryDeviceDetail&version=1&product_id=lsibd9&device_name=no001		
响应示例	<pre>{ "data": { "device_name": "no001", "product_id": "9MaNe52pNO", "product_name": "空气净化器", "active_time": "2020-06-19T08:11:27.801Z", "created_time": "2020-06-19T06:09:22.550Z", "desc": "设备 1", "ip": "192.168.200.139", "last_time": "2020-06-19T09:48:15.027Z", "node_type": 1, "protocol": 2, "sec_key": "UQd3K9lXR/EbLXeJc50lJfvvkTVdu5uFgbfz48/fl5k=", "status": 3, "template_id": "6406f2ee9641f20035c53b12", "template_field": {}, "scene": ["a25087f46df040ef0acfd115"] }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

	}
--	---

1.4.2.6. 设备状态查询

方法	GET			
路径 URI	/common?action=DeviceStatus&version=1			
请求头				
URL 参 数	product_id	string	必填	产品 ID
	device_name	string	必填	设备名称
请求体	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回业务数据	
	data.status	int	设备状态 1-未激活 2-在线 3- 离线	
请求示 例	GET /common?action=DeviceStatus&version=1&product_id=9MaNe52pNO& device_name=no001			
响应示 例	{ "data": { status: 1 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }			

1.4.2.7. 设备状态记录查询

方法	GET			
路径 URI	/common?action=DeviceStatusHistory&version=1			
请求头				
URL 参数	product_id	string	必填	产品 ID
	device_name	string	必填	设备名称
	start_time	string	必填	查询起始时间，毫秒时间戳
	end_time	string	必填	查询结束时间，毫秒时间戳
	offset	string	可选	请求起始位置，默认 0
	limit	string	可选	每次请求记录数，默认 10，范围[1, 100]
请求体	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回业务数据	
	data.list	array	设备状态历史数据集合，如下的 l 表示 list 数组的单个对象标识	
	l.status	int	设备状态 0-离线 1-在线	
	l.time	date	时间戳	
	data.meta	object	分页信息	
	data.meta.limit	int	每次请求记录数	
	data.meta.offset	int	请求记录起始位置	
请求示例	GET url/common?action=QueryDeviceStatusHistory&version=1&product_id=9MaNe52pNO&device_name=no001&start_time=1592795951065&end_time=1592795971065			
响应示例	{ "data": { "list": [{ "status": 0, "time": 1592398421297, }], "meta": {			

	<pre> "limit": 10, "offset": 0 } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	--

1.4.2.8. 设备属性设置

方法	POST			
路径 URI	/common?action=SetDeviceProperty&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	params	object	必填	设置的属性值, 数据格式为 json 对象, 形式为 key:value, key 为属性功能点标识, value 为属性值, 取值符合物模型定义的数据类型和取值范围, 例如 { "switch": true }
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	--		调用成功时, 返回的业务数据
	<u>data.id</u>	string		设备端回复消息 id
	data.code	int		设备端回复响应码
	data.msg	string		设备端回复响应消息

请求示例	<pre>{ "product_id": "9MaNe52pNO", "device_name": "no001", "params": { "switch": true, // bool "text": "hello", // string "humidity": 12 , // int32 "number": 1564448722123, // int64 "temperature": 30.2 // float "lng": 3.1234567890123456789, // double "type": 1, // enum "error": 256, // bitmap "event": { // struct "a": 1, "b": true } } }</pre>
响应示例	<pre>{ "data": { "id": "155", "code": 200, "msg": "success" }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>

1.4.2.9. 设备属性获取

方法	POST			
路径 URI	/common?action=QueryDevicePropertyDetail&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	device_name	string	必填	设备唯一标识
	product_id	string	必填	产品 ID
	params	array	必填	功能点标识数组, example: ["light","model"]
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	--		调用成功时, 返回的业务数据
请求示例	{ "product_id":"B7EEW578EbRg5Y4K", "device_name":"device3", "params":["light","model"] }			
响应示例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true, "data":{ "light":1, "model":1 } }			

1.4.2.10. 设备属性期望值设置

方法	POST			
路径 URI	/common?action=SetDeviceDesiredProperty&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	params	object	必填	设置的属性期望值, 数据格式为 json 对象, 形式为 key:value, key 为属性功能点标识, value 为属性值, 取值符合物模型定义的数据类型和取值范围, 例如{ "switch": true }
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
请求示例	{ "product_id": "9MaNe52pNO", "device_name": "no001", "params": { "switch": true, // bool "text": "hello", // string "humidity": 12 , // int32 "number": 1564448722123, // int64 "temperature": 30.2 // float "lng": 3.1234567890123456789, // double "type": 1, // enum }			

	<pre> "error": 256, // bitmap "event": { // struct "a": 1, "b": true } } </pre>
响应示例	<pre> { "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>

1.4.2.11. 设备属性期望值查询

方法	POST			
路径 URI	/common?action=QueryDeviceDesiredProperty&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	params	array	必填	查询期望值的功能点标识集合， 参数不传默认查询所有属性期望，例如["switch", "temperature"]
响应参 数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功

	data	——	调用成功返回业务数据
	data.params	object	属性功能点期望值
	data.params. identify	object	功能点标识为对象 key
	data.params. identify.value	string	期望值值
请求示例	<pre>{ "product_id": "9MaNe52pNO", "device_name": "no001", "params": ["switch", "temperature",] }</pre>		
响应示例	<pre>{ "data": { "params": { "switch": { "value": "on" }, "temperature": { "value": 23 } } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.2.12. 设备属性期望删除

方法	POST			
路径 URI	/common?action=DeleteDeviceDesiredProperty&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	params	object	必填	删除的属性期望, 数据格式为 json 对象, 形式为 key:value, key 为属性功能点标识, value 为空对象{}
响应参 数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
请求示 例	{ "product_id": "12909", "device_name": "no001", "params": { "temperature": {}, // 删除期望值 } }			
响应示 例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true			

	}
--	---

1.4.2.13. 设备属性最新数据查询

方法	GET			
路径 URI	/common?action=DevicePropertyData&version=1			
请求头	Content-Type : application/json			
URL 参数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
请求体参数	无			
响应参数	code	string	错误码, code 为 “0” 代表请求成功	
	msg	string	错误消息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时, 返回的业务数据	
	data.list	array	设备最新数据集合, 如下的 1 表示 list 数组的单个对象标识	
	l.identifier	string	功能点标识	
	l.time	string	上报时间, 毫秒时间戳	
	l.value	string	功能点上报值, json 字符串	
	l.data_type	string	数据类型 int32、int64、float、double、enum、bool、string、struct、bitMap	
	l.access_mode	string	读写类型	
	l.expect_value	string	期望值, json 字符串, 属性功能	

			点具有该字段
	l.name	string	功能点名称
	l.description	string	功能描述
请求示例	GET /common?action=DevicePropertyData&version=1&product_id=9MaNe52pNO&device_name=no001		
响应示例	<pre>{ "data": { "list": [{ { "access_mode": "读写", "data_type": "int32", "description": "二氧化碳", "expect_value": "800", "identifier": "CO2", "name": "二氧化碳", "time": "1592797444539", "value": "600" }, { "access_mode": "读写", "data_type": "bool", "description": "关", "expect_value": "true", "identifier": "fan", "name": "风扇", "time": "1592797444539", "value": "false" }] } }</pre>		

	<pre>} } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	---

1.4.2.14. 设备属性功能点历史数据查询

方法	GET			
路径 URI	/common?action=DevicePropertyHistory&version=1			
请求头				
URL 参 数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
	identifier	string	必填	属性功能点标识
	start_time	string	必填	查询起始时间，毫秒时间戳
	end_time	string	必填	查询结束时间，毫秒时间戳
	offset	string	可选	请求起始位置，默认 0
	limit	string	可选	每次请求记录数，默认 10，范围[1, 100]
请求体 参数	无			
响应参 数	code	string		错误码，code 为“0”代表请求成功
	msg	string		错误消息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	—		调用成功时，返回的业务数据

	data.list	array	设备属性记录集合，如下的1表示 list 数组的单个对象标识
	l.value	string	属性功能点上报值
	l.time	string	属性功能点上报时间
	data.meta	object	分页信息
	data.meta.limit	int	每次请求记录数
	data.meta.offset	int	请求记录起始位置
请求示例	GET /common?action=DevicePropertyHistory&version=1&product_id=9MaNe52pNO&device_name=no001&identifier=fan&start_time=1615342778414&end_time=1615342898096		
响应示例	<pre>{ "data": { "meta": { "offset": 0, "limit": 10 }, "list": [{ "time": "1639380798206", "value": "qweasd" }, { "time": "1639380795091", "value": "qweasd" }] }, }</pre>		

	<pre> "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	---

1.4.2.15. 设备事件功能点（单个）历史数据

方法	GET			
路径 URI	/common?version=1&action=DeviceEventHistory			
请求头				
URL 参 数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
	identifier	string	可选	事件功能点标识
	start_time	string	必填	查询起始时间，毫秒时间戳
	end_time	string	必填	查询结束时间，毫秒时间戳
	offset	string	可选	请求起始位置，默认 0
	limit	string	可选	每次请求记录数，默认 10，范围[1, 100]
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.list	array	设备事件记录历史数据集合，如下的 l 表示 list 数组的单个对象标识	
	l.event_type	int	事件类型 1-信息 2-告警 3-故	

			障
	l.identifier	string	事件功能点标识
	l.name	string	事件功能点名称
	l.time	string	事件功能点上报时间
	l.value	string	事件功能点上报值, json 字符串
	data.meta	object	分页信息
	data.meta.limit	int	每次请求记录数
	data.meta.offset	int	请求记录起始位置
请求示例	GET /common?version=1&action=DeviceEventHistory&product_id=9MaNe52pNO&device_name=no001&start_time=1592811019119&end_time=1592811198213		
响应示例	<pre>{ "data": { "meta": { "offset": 0, "limit": 10 }, "list": [{ "time": "1639381959167", "value": "{\"a\":11}", "event_type": 1, "identifier": "sj01", "name": "sj01" }] }, }</pre>		

	<pre> "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	---

1.4.2.16. 设备服务执行记录查询

方法	GET			
路径 URI	/common?action=DeviceServiceHistory&version=1			
请求头				
URL 参 数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
	start_time	string	必填	查询起始时间，毫秒时间戳
	end_time	string	必填	查询结束时间，毫秒时间戳
	offset	string	可选	请求起始位置，默认 0
	limit	string	可选	每次请求记录数，默认 10，范围[1, 100]
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.list	array	设备服务执行记录数据集合，如下的 l 表示 list 数组的单个对象标识	
	list.request_time	string	调用服务的时间	

	list.function_name	string	功能名称
	list.identifier	string	标识符
	list.type	string	服务调用类型, 0 为同步, 1 为异步
	list.request_body	string	服务调用时的请求参数
	list.response_time	string	返回时间
	list.response_body	string	输出参数, json string
	list.code	string	执行结果 code, 200: 执行成功, 0: 未执行, 其他: 执行异常
	list.msg	string	返回消息
	data.meta	object	分页信息
	data.meta.limit	int	每次请求记录数
	data.meta.offset	int	请求记录起始位置
请求示例	GET /common?action=DeviceServiceHistory&version=1&product_id=7ubgKilvhm&device_name=sb0011&start_time=1639050724393&end_time=1639381724393		
响应示例	<pre>{ "data": { "meta": { "offset": 0, "limit": 10 }, "list": [{</pre>		

	<pre>"request_time":"1639383257642", "function_name":"s1", "identifier":"s1", "type":1, "request_body":{"\id\":"2\","\params\":{"identifier\":"s1","\input\":{"b\":"2}},\version\":"1.0\"}," "response_time":"0", "response_body":"", "code":0, "msg":"" }] } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	---

1.4.2.17.设备操作记录

方法	GET			
路径 URI	/common?action=DeviceOperateHistory&version=1			
请求头	Content-Type : application/json			
URL 参数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
	start_time	string	必填	开始时间，毫秒时间戳
	end_time	string	必填	结束时间，毫秒时间戳
	limit	string	可选	每次请求记录数，默认 10，范围 [1, 100]
	offset	string	可选	请求起始位置，默认 0

请求体参数	无		
响应参数	code	string	错误码, code 为 “0” 代表请求成功
	msg	string	错误消息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时, 返回的业务数据
	data.list	array	设备操作记录集合, 如下的 l 表示 list 数组的单个对象标识
	l.request_time	string	请求时间
	l.type	int	请求类型 0-写 1-读
	l.request_body	object	请求内容 k=>v 形式, k 为属性功能点标识, v 为功能点设置值
	l.response_time	string	响应时间
	l.response_body	object	响应结果, 设备回复响应中的 msg 字段
	l.response_body.id	string	响应结果中的 id
	l.response_body.code	int	响应结果中的 code
	l.response_body.msg	string	响应结果中信息
	l.code	int	执行结果 code, 200: 执行成功
	l.msg	string	返回消息
	data.meta	object	分页信息

	data.meta.limit	int	每次请求记录数
	data.meta.offset	int	请求记录起始位置
请求示例	GET /common?action=DeviceOperateHistory&version=1&product_id=9MaNe52pNO&device_name=no001 &start_time=1592398386297&end_time=1592398422297		
响应示例	<pre>{ "data": { "list": [{ "request_time": "1639382709533", "type": 0, "request_body": "{\"tt\":1}", "response_time": "1639382709570", "response_body": { "id": "1", "code": 200, "msg": "success" }, "code": 200, "msg": "success" }], "meta": { "offset": 0, "limit": 10 } } }</pre>		

	<pre> }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	--

1.4.2.18. 设备服务调用

方法	POST			
路径 URI	/common?action=CallService&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	string	必填	设备名称
	product_id	string	必填	产品 ID
	identifier	string	必填	服务型功能点标识
	params	object	必填	输入参数的键值对，输入参数的唯一标识做键
响应参 数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	——		调用成功返回业务数据
请求示 例	{ "product_id": "B7EEW578EbRg5Y4K", "device_name": "device3", "identifier": "light", "params": { "Power1": "1",			

	<pre> "WF1": "2" } } </pre>
响应示例	<pre> { "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true, "data": { "result1": "1", "result2": "2" } } </pre>

1.4.2.19. 设备链路日志查询

方法	GET			
路径 URI	/common?action=DeviceTraceLog&version=1			
请求头				
URL 请求参数	product_id	string	可选	产品 id
	device_name	string	可选	设备名称
	start_time	string	必填	查询起始时间，毫秒时间戳
	end_time	string	必填	查询结束时间，毫秒时间戳
	offset	string	可选	请求起始位置，默认 0
	limit	string	可选	每次请求记录数，默认 10，范围[1, 100]

	log_type	string	可 选	业务类型编码，参考备注
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.list	array	设备链接日志记录数据集合	
	data.meta	object	分页信息	
	data.meta.limit	int	每次请求记录数	
	data.meta.offse t	int	请求记录起始位置	
请求示 例	GET /common?action=DeviceTraceLog&version=1start_time=1639050724393&end_time=1639381724393			
响应示 例	{ "data": { "list":[{ "type":"1.1", "start_time":"2021-12-10T10:19:33.325Z", "pid":"7ubgKilvhm", "status":"200", "trace_id":"ab2fc05759a211ecbfd023bbbfff50a4", "end_time":"2021-12-10T10:19:33.325Z", "env_type":"UNKNOWN",			

	<pre> "uid": "5", "device_name": "400A002090011001", "content": "{ \"protocol\": \"MQTT\", \"offline_time\": \"2021-12-10 18:19:33.325\", \"offline_reason\": \"CloseByPeer\" }", "create_time": "2021-12-10 18:19:33", "message_status": "0", "type_mean": "设备行为", "status_mean": "成功" }], "meta": { "total": 26, "limit": "11", "offset": "0" } } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
备注	业务类型对照表: { '1.1': '设备行为', '1.2': '上行消息', '1.3': '下行消息', '2.1': '物模型调用', '3.1': '数据存储', '4.1': '规则引擎', '5.1': 'MQ 推送', '6.1': 'HTTP 推送', '7.1': '开放 API',

	'8.1': 'RDS 存储', '9.1': '应用长连接' }
--	---

1.4.2.20. 设备上下行消息查询

方法	GET			
路径 URI	/common?action=DeviceMessage&version=1&message_id=fe8e158c5cab11ecbfd05180177bf746			
请求头				
URL 参 数	message_id	string	必填	消息 id 通过接口【设备链路日志 查询】获取（只存在部分类型的 日志信息中）
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据， data 为 base64 字符串，可转义	
请求示 例	GET /common?action=DeviceMessage&version=1&message_id=fe8e158c5cab11ecbfd05180177bf746			
响应示 例	{ "data": "eyJpZCI6IjE2Mzk0NjU0MzIzOTMiLCJjb2RlIjoyMDAsIm1zZyI6InN1Y2Nlc3MifQ==",, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true			

	}
--	---

1.4.2.21. 设备转移

方法	POST			
路径 URI	/common?action=MoveProductDevice&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	array	必填	被转移设备的名称数组，如["name_1", "name_2"]
	product_id	string	必填	产品 ID
	target_user_tel	string	必填	接收人电话
响应参 数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	—		调用成功时，返回的业务数据
请求示 例	{ "product_id": "IOoSyVbY8q", "device_name": ["SB00009"], "target_user_tel": "17830021227" }			
响应示 例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true,			

	<pre>"data": { "move_id": "1317465" }</pre>
--	---

1.4.2.22.设备批量命令下发

方法	POST			
路径 URI	/common?action=DeviceBatchOrder&version=1			
请求头	Content-Type : application/json			
URL 参 数	无			
请求体 参数	device_name	array	必填	设备名称组成的数组
	product_id	string	必填	产品 ID，只能对同一产品下的多个设备下发
	type	string	必填	可选['SET_PROPERTY', 'GET_PROPERTY', 'CALL_SERVICE'], 依次为属性设置、属性获取、服务调用
	mode	number	必填	命令模式，1：串行；2：并行。
	identifier	string		参数 type 值为 CALL_SERVICE 时必填，服务型功能点标识
	params	object 或者 array	必填	根据参数 type 值参考对应接口的 params 进行构造： SET_PROPERTY： 设备属性设置 GET_PROPERTY： 设备属性获取 CALL_SERVICE： 设备服务

				调用
响应参数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	——		调用成功返回业务数据
请求示例	{ "type": "SET_PROPERTY", "mode": 1, "product_id": "fcvPpxC22R", "device_name": ["test9981121"], "params":{"a":1} }			
响应示例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true, "data":{ "successCount": 0, // 命令成功数 "errorCount": 1, // 如果是并行命令则表示命令失败数，如果是串行命令表示剩余未下发命令的设备数 "errorDevs": [] // 命令下发失败的设备名称, 只有并行命令时有值，串行命令时无数据值 } }			

1.4.3. 物模型管理

1.4.3.1. 物模型查询

方法	GET			
路径 URI	/common?action=QueryThingModel&version=1&product_id=lsibd9			
请求头				
URL 参 数	product_id	string	必 填	产品 id
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	object	调用成功时，返回的业务数据	
	data.properties	array	数组对象 属性功能点	
	p.functionMod e	string	功能点类型，定值'property'	
	p.identifier	string	属性唯一标识符（产品下唯一）	
	p.name	string	属性名称	
	p.desc	string	属性描述	
	p.accessMode	string	"属性读写类型：只读（r）或读写（rw）	
	p.functionType	string	是否是标准功能点，自定义（u）/系统（s）/标准（st）	
	p.dataType	object	属性功能点数据	
	p.dataType.typ	string	属性类型：int32（32 位整数）、int64（64	

e		位整数)、float (单精度浮点)、double (双精度浮点型)、string (字符串)、date (String 类型 UTC 秒)、bool (true 或 false)、enum (int 类型)、bitMap (位图)、date (int64 类型 UTC 时间戳毫秒)、struct (结构体类型)、array (数组)
p.dataType.spe cs	object	属性功能点数据
data.events	array	数组对象 事件功能点
e.functionMode	string	功能点类型, 定值'event'
e.identifier	string	事件唯一标识符
e.name	string	事件名称
e.desc	string	事件描述
e.type	string	事件类型 (info、alert、error)
e.fuctionType	string	是否是标准功能点, 自定义 (u) /标准 (s)
e.outputData	array	参数
e.outputData.id entifier	string	参数唯一标识符
<u>e.outputData.n ame</u>	string	参数名称
e.outputData.d ataType	object	参数数据
e.outputData.d ataType.type	string	属性类型: int32 (32 位整数)、int64 (64 位整数)、float (单精度浮点)、double (双精度浮点型)、string (字符串)、bool (true 或 false)、enum (int 类型)、bitMap (位图)、date (int64 类型 UTC 时间戳毫秒)、struct (结构体类型)、

		array (数组)
e.outputData.dataType.specs	object	功能点数据
services	array	数组对象 服务功能点
s.functionMode	string	功能点类型, 定值'service'
s.identifier	string	服务唯一标识符 (产品下唯一)
s.name	string	服务名称
s.desc	string	服务描述
s.callType	string	调用方式,同步(s)/异步(a)
s.fuctionType	string	功能点类型, 自定义 (u) /系统 (s)
s.input	array	输入参数
s.input.identifier	string	参数唯一标识符
<u>s.input.name</u>	string	参数名称
s.input.dataType	object	参数数据
s.input.dataType.type	string	属性类型: int32 (32 位整数)、int64 (64 位整数)、float (单精度浮点)、double (双精度浮点型)、string (字符串)、bool (true 或 false)、enum (int 类型)、bitMap (位图)、date (int64 类型 UTC 时间戳毫秒)、struct (结构体类型)、array (数组)
s.input.dataType.specs	object	参数功能点数据
s.output	array	输出参数
s.output.identifier	string	参数唯一标识符

	<u>s.output.name</u>	string	参数名称
	s.output.dataType	object	参数数据
	s.output.dataType.type	string	属性类型: int32 (32 位整数)、int64 (64 位整数)、float (单精度浮点)、double (双精度浮点型)、string (字符串)、bool (true 或 false)、enum (int 类型)、bitMap (位图)、date (int64 类型 UTC 时间戳毫秒)、struct (结构体类型)、array (数组)
	s.output.dataType.specs	object	参数功能点数据
请求示例	GET /common?action=QueryThingModel&version=1&product_id=lsibd9		
响应示例	<pre>{ "data": { "properties": [{ "name": "模式", "identifier": "model", "functionType": "u", "functionMode": "property", "desc": "", "accessMode": "rw", "dataType": { "type": "enum", "specs": { "1": "模式 1", "2": "模式 2" } } }] } }</pre>		


```
    }
  }
],
"events": [
  {
    "name": "test",
    "identifier": "test",
    "functionType": "u",
    "functionMode": "event",
    "desc": null,
    "eventType": "info",
    "outputData": [
      {
        "dataType": {
          "type": "bool",
          "specs": {
            "false": "关",
            "true": "开"
          }
        },
        "name": "开关",
        "identifier": "switch"
      }
    ]
  }
]
}

"requestId": "a25087f46df04b69b29e90ef0acfd115",
"success": true
```

	}
--	---

1.4.4. 文件管理

1.4.4.1. 设备文件上传

方法	POST			
路径 URI	/common?action=CreateDeviceFile&version=1			
请求头	Content-type: multipart/form-data			
URL 参数	无			
请求体参数	device_name	string	必填	设备名称 ,参数需构造在同一个 form-data 中
	product_id	string	必填	产品 ID ,参数需构造在同一个 form-data 中
	file	file	必填	上传的图片文件(目前支持 JPG、JPEG、PNG、BMP、GIF、WEBP、TIFF、TXT)
	md5	string	必填	文件的 MD5
	size	number	必填	文件大小(字节)
响应参数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	-		调用成功时, 返回的业务数据
	data.fid	string		文件上传成功后返回的文件 ID
请求示例	{			

	<pre>"device_name": "device_name", "product_id": "qwdfbht", "md5": "f55c2e86ab864b64a6d939fbe3a7d65f", "size": 12546, "file": file }</pre>
响应示例	<pre>{ "data": { "fid": "434fa51a170942c8a291be1e4b229582" }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>

1.4.4.2. 查看下载设备文件接口

方法	GET			
路径 URI	/common?action=GetDeviceFile&version=1			
请求头				
URL 参数	id	string	必填	文件 id
请求体	无			
响应参数	file	file	需要下载的文件	
请求示例	GET /common?action=GetDeviceFile&version=1&id=43bb54ac673f48c88300fa6e6d3c9481			
响应示例	file			

1.4.4.3. 文件删除接口

方法	POST
路径 URI	/common?action=DeleteDeviceFile&version=1

请求头	Content-type: multipart/form-data			
URL 参数	无			
请求体参数	id	string	必填	文件 ID
响应参数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	—		调用成功时，返回的业务数据
请求示例	{ id: 43bb54ac673f48c88300fa6e6d3c9481 }			
响应示例	{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "code": "0", "msg": "success", "data": null, "success": true }			

1.4.5. 分组管理

1.4.5.1. 分组创建

方法	POST
路径 URI	/common?action=GroupCreate&version=1
请求头	Content-Type : application/json
URL 参数	无

请求体 参数	name	string	必填	分组名称
	desc	string	可选	分组描述
响应参 数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	--		调用成功时，返回的业务数据
	data.group_id	string		分组 ID
请求示 例	{ "name": "黄河大道东", "desc": "group 1" }			
响应示 例	{ "data": { "group_id": "uwYqby" }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }			

1.4.5.2. 分组删除

方法	POST			
路径 URI	/common?action=OuterDeleteGroup&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	group_id	string	必填	分组 ID

响应参数	code	string	调用失败时，返回的错误码
	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	--	调用成功时，返回的业务数据
请求示例	<pre>{ "group_id": "3UfAWD" }</pre>		
响应示例	<pre>{ "data": null, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.5.3. 分组编辑

方法	POST			
路径 URI	/common?action=OuterUpdateGroup&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	group_id	string	必填	分组 ID
	tag	object	可选	标签信息
	desc	string	可选	分组描述
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	--	调用成功时，返回的业务数据	

	data.group_id	string	分组 ID
请求示例	<pre>{ "group_id": "3UfAWD", "tag": {"key11": "dkmclg"}, #标签的键值对 "desc": "描述" }</pre>		
响应示例	<pre>{ "data": { "group_id": "diVGB3" }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.5.4. 分组列表

方法	GET			
路径 URI	/common?action=OuterGroupList&version=1			
请求头				
URL 参数	name	string	可选	分组名称
	key	string	可选	标签 key (key、value 需成对出现, 否则没有效果)
	value	string	可选	标签 value (key、value 需成对出现, 否则没有效果)
	offset	string	可选	请求记录起始位置, 默认 0
	limit	string	可选	每次请求记录数, 默认 10
请求体	无			
响应参数	code	string	调用失败时, 返回的错误码	
	msg	string	调用失败时, 返回的错误信息	

	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	–	调用成功时, 返回业务数据
	data.list	array	分组信息集合, 如下的 l 表示 list 数组的单个对象标识
	l.name	string	分组名称
	l.group_id	string	分组 id
	l.key	string	分组 key
	l.tag	object	标签信息, 键值对
	l.created_time	string	创建时间
	l.device_count	int	设备数
	data.meta	object	分页信息
	data.meta.limit	int	每次请求记录数
	data.meta.offset	int	请求记录起始位置
	data.meta.total	int	记录总数
请求示例	GET /common?action=OuterGroupList&version=1		
响应示例	<pre>{ "data": { "list": [{</pre>		

	<pre>"name": "xq_group1", "group_id": "qf6nAD", "key": "ZDM0MzA4MTA3MjQ4Nzd1YzZjOGJlYzU1YmUwZTNhMmY=", "tag": { "xq": "123" }, "created_time": "2020-08-13T01:49:17694", "device_count": 2 }], "meta": { "limit": 10, "offset": 0, "total": 1 } }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	--

1.4.5.5. 分组详情

方法	GET			
路径 URI	/common?action=OuterGroupDetail&version=1			
请求头				
URL 参数	group_id	string	必填	分组 ID
请求体	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	

	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时, 返回业务数据
	data.activate_count	string	激活设备数
	data.online_count	string	在线设备数
	<u>data.name</u>	string	分组名称
	data.group_id	string	分组 ID
	data.key	string	分组 key
	data.tag	object	分组标签
	data.desc	string	分组描述
	data.device_count	string	设备数量
	data.create_time	string	创建时间
请求示例	GET /common?action=OuterGroupDetail&version=1		
响应示例	<pre>{ "data": { "activate_count": 0, "online_count": 0, "name": "xq_group1", "group_id": "qf6nAD", "key": "ZDM0MzA4MTA3MjQ4NzdlYzZjOGJlYzU1YmUwZTNhMmY=", "tag": { "xq": "123" }, "desc": "123", } }</pre>		

	<pre> "created_time": "2020-08-13T01:49:17.694Z", "device_count": 2 } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	--

1.4.5.6. 分组设备添加

方法	POST			
路径 URI	/common?action=OuterAddGroupDevice&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	group_id	string	必填	分组 ID
	product_id	string	必填	产品 ID
	devices	array	必填	需要添加的设备集合
响应参数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	--		调用成功时，返回的业务数据
	data.error_data	array		添加失败的错误信息集合，如下的 e 表示 error_data 数组的单个对象标识
	e. device_name	string		添加失败的设备集合
	e. cause	string		添加失败原因
请求示例	{ "group_id": "Z1Pdei", "product_id": "XVlg5CCSSj",			

	<pre>"devices": ["dev1", "dev2"] }</pre>
响应示例	<pre>{ "data": { "group_id": "qf6nAD", "devices": ["dev1", "dev2"] }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>

1.4.5.7. 分组设备移除

方法	POST			
路径 URI	/common?action=OuterRemoveGroupDevice&version=1			
请求头	Content-Type : application/json			
URL 参数	无			
请求体 参数	group_id	string	必填	分组 ID
	product_id	string	必填	产品 ID
	devices	array	必填	需要移除的设备集合
响应参数	code	string		调用失败时，返回的错误码
	msg	string		调用失败时，返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	--		调用成功时，返回的业务数据
	data.error_data	array		移除失败的错误信息集合，如下的 e 表示 error_data 数组的单个对象标识
	e.	string		移除失败的设备集合

	device_name		
	e. cause	string	移除失败原因
请求示例	<pre>{ "group_id": "Z1Pdei", "product_id": "XVlg5CCSSj", "devices": ["dev1", "dev2"] }</pre>		
响应示例	<pre>{ "data": { "group_id": "qf6nAD", "devices": ["dev1", "dev2"] }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.6. 用户管理

1.4.6.1. 用户下产品数量统计

方法	GET
路径 URI	/common?action=ProductStatistics&version=1
请求头	
URL 参数	无
请求体参数	无

响应参数	code	string	调用失败时，返回的错误码
	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据
	data.product_agg regate.product_c ount	number	产品数量
	data.product_agg regate.ind_agg	array	数组对象，行业类型统计，如下的 ind 表示 ind_agg 数组的单个对象标识
	ind._id	string	行业 ID（自动生成）
	ind.count	string	行业类别下产品数量
	<u>ind.name</u>	string	行业名称
	data.product_agg regate.pt_agg	array	数组对象，协议类型统计，如下的 pt 表示 pt_agg 数组的单个对象标识
	pt._id	string	协议 ID（自动生成）
	pt.count	string	同一协议下产品数量
	<u>pt.name</u>	string	协议名称
	data.product_agg regate.net_agg	array	数组对象，联网类型统计，如下的 net 表示 net_agg 数组的单个对象标识
	net._id	string	联网类型 ID（自动生成）
	net.count	string	同一联网类型下产品数量
	<u>net.name</u>	string	联网类型名称
	data.product_agg regate.model_agg	array	数组对象，物模型类型统计，如下的 model 表示 model_agg 数组的单个对象标识
	model._id	string	物模型类型 ID（自动生成）

	model.count	string	物模型类型下产品数量
	<u>model.name</u>	string	物模型类型名称
请求示例	GET /common?action=ProductStatistics&version=1		
响应示例	<pre>{ "data": { "product_aggregate": { "ind_agg": [// 行业类别 { "_id": "1", "count": 7, "name": "智慧城市" }], "pt_agg": [// 协议类型 { "_id": 'sub', "count": 1, "name": '网关子设备' }, { "_id": 4, "count": 2, "name": 'Lwm2m' }, { "_id": 2, "count": 4, "name": "MQTT" }] } } }</pre>		

	<pre>], "net_agg":[// 联网型类型 { "_id":"1", "count":0, "name":"其他" }, { "_id":"2", "count":0, "name":"蜂窝" }, { "_id":"3", "count":5, "name":"wifi" }, { "_id":"4", "count":0, "name":"以太网" }, { "_id":"5", "count":1, "name":"NB" }], "model_agg":[// 物模型类型</pre>
--	---

	<pre>{ { "_id": "1", "count": 2, "name": "标准" }, { "_id": "2", "count": 5, "name": "自定义" } }] }, "product_count": 7 // 总数 } } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	--

1.4.6.2. 账户下文件列表查询

方法	GET			
路径 URI	/common?action=GetDeviceFilesList&version=1			
请求头				
URL 参数	offset	number	可选	请求起始位置，默认 0
	limit	number	可选	每次请求记录数，默认 10
请求体参数	无			

响应参数	code	string	调用失败时，返回的错误码
	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据
	data.meta.total	number	文件数量
	data.meta.limit	number	每次请求的数据长度
	data.meta.offset	number	偏移量
	data.list.fid	string	文件 ID
	<u>data.list.name</u>	string	文件名称
	data.list.file_size	number	文件大小
	data.list.ct	string	文件创建时间
	data.list.device_name	string	文件所属设备名称
	data.list.product_id	string	文件所属设备的产品 ID
请求示例	GET /common?action=GetDeviceFilesList&version=1		
响应示例	<pre>{ "data": { "meta": { "limit": 10, "offset": 0, "total": 1 } } }</pre>		

	<pre> }, "list": [{ "fid": "98cfa6be79574f7eab98eb7b5222911a", "name": "28fpf.png", "file_size": 138683, "ct": "2020-12-16T09:30:18.419Z", "device_name": "ap-test-008", "product_id": "Bs1f6s5bhKP7rmfO" }] } "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	---

1.4.6.3. 用户文件存储空间查询

方法	GET		
路径 URI	/common?action=GetDeviceFileSpace&version=1		
请求头			
URL 参数	无		
请求体参数	无		
响应参数	code	string	调用失败时，返回的错误码
	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功

	data	—	调用成功时，返回的业务数据
	data.useSize	number	用户已使用的空间
	data.hasSize	number	用户剩余空间
	data.totalSize	number	用户分配的总空间
请求示例	GET /common?action=GetDeviceFileSpace&version=1		
响应示例	<pre>{ "data": { "useSize": 138683, "hasSize": 1073603141, "totalSize": 1073741824 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.6.4. 用户设备文件数量查询

方法	GET			
路径 URI	/common?action=GetDeviceFileCount&version=1&device_name=ap-test-008&product_id=Bs1f6s5bhKP7rmfO			
请求头				
URL 请求参数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
请求体参数	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	

	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时, 返回的业务数据
	data.upperLimit	number	设备允许的最大文件数量
	data.filesTotal	number	设备已存在的文件数量
请求示例	GET /common?action=GetDeviceFileCount&version=1&device_name=ap-test-008&product_id=Bs1f6s5bhKP7rmfO		
响应示例	<pre>{ "data": { "upperLimit": 10, "filesTotal": 1 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.7. LwM2M-IPSO 管理

1.4.7.1. 设备 IPSO 最新数据查询

方法	GET			
路径 URI	/common?action=QueryLatestData&version=1			
请求头	authorization - 鉴权信息			
URL 参数	imei	string	必填	设备 IMEI 号
请求体参数	无			
响应参数	code	string	调用失败时, 返回的错误码	

	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据
	data.list	array	资源数据数组
	data.list[x].ds_id	string	资源名称，ds_id 为 {obj_id}_{obj_inst_id}_{res_id} 组合。 例如：obj_id:3200, obj_inst_id:0, res_id:5501，那么这个 datastream_id 就为 3200_0_5501
	data.list[x].value	string	资源当前值，只有上报过数据的才有最新值
	data.list[x].time	string	更新时间
请求示例	GET /common?action=QueryLatestData&version=1&imei=86101704647522		
响应示例	<pre>{ "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true "data": { "list": [{ "ds_id": "3_0_0", "value": "NBDeviceSimulator", "time": "1637115467787" }, { "ds_id": "3_0_1", "value": "模型 500", "time": "1637115467786" }] }</pre>		

	<pre>}, { "ds_id": "3_0_2", "value": "LT-500-000-0001", "time": "1637115467787" } } }</pre>
--	---

1.4.7.2. 设备 IPSO 资源历史数据查询

方法	GET			
路径 URI	/common?action=QueryHistoryData&version=1			
请求头	authorization - 鉴权信息			
URL 参数	imei	string	必填	设备 IMEI 号
	ds_id	string	必填	资源名称, ds_id 为 {obj_id}_{obj_inst_id}_{res_id}组合, 查询时必须指定到实例具体资源。例如: obj_id:3200, obj_inst_id:0, res_id:5501, 那么这个 ds_id 就为 3200_0_5501
	start_time	string		查询起始时间, 毫秒时间戳
	end_time	string		查询结束时间, 毫秒时间戳
	offset	string		请求起始位置, 默认 0
	limit	string		单次请求记录数, 默认 10, 范围 [1.500]
请求体参数	无			
响应参数	code	string	调用失败时, 返回的错误码	

	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据数组
	data.list	array	资源数据数组
	data.list[x].value	string	资源值
	data.list[x].time	string	更新时间
请求示例	GET /common?action=QueryHistoryData&version=1&imei=861017046475226&ds_id=3303_0_5601&start_time=1634632350500&end_time=1634632350509&limit=10&offset=0		
响应示例	<pre>{ "code": "0", "msg": "success", "data": { "meta": { "limit": 0, "offset": 10, }, "list": [{ "value": "NBDeviceSimulator", "time": "1637115467788" }, { "value": "模型 500", "time": "1637115467789" }, {</pre>		

	<pre>"value": "LT-500-000-0001", "time": "1637115467790" } }</pre>
--	--

1.4.8. 超管接口

1.4.8.1. 产品详情

方法	GET			
路径 URI	/common?action=SAProductDetail&version=1&product_id=10132			
请求头				
URL 请 求参数	product_id	string	必填	产品 id
请求体 参数	无			
响应参 数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.device_number	number	设备数量	
	data.product_id	string	产品 ID	
	<u>data.name</u>	string	产品名称	
	data.desc	string	产品描述	

	data.network	string	联网方式
	data.category	list<string>	产品所属分类 ID
	data.protocol	number	协议类型 1-泛协议 2-MQTT 3-CoAP 4-LwM2M 5-HTTP 为空时表示产品节点类型为子设备
	data.create_time	string	创建时间
	data.ind_title	string	产品行业名称
	data.prod_ind	string	产品行业编码
	data.uid	string	产品用户 ID
	data.sec_key	string	产品权限 key
请求示例	GET /common?action=SAProductDetail&version=1&product_id=10132		
响应示例	<pre>{ "data": { "device_number":0, "protocol":2, "create_time":"2021-12-06T08:28:56.762Z", "product_id":"wjrJjSRtsG", "network":"1", "category":["138"], "ind_title":"智慧城市", "prod_ind":"1", "uid":37782, "sec_key":"vC3eX2gr8mapsNZ1zYVmtsFRMYaX953GbJiK1cAQTp4=", "desc": "", "name":"物模型" }, }</pre>		

	<pre>"requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>
--	---

1.4.8.2. 产品列表

方法	GET			
路径 URI	/common?action=SAProductList&version=1			
请求头				
URL 请求参数	name	string	可选	产品名称
	manufacturer	string	可选	厂商名称
	protocol	string	可选	接入协议, 可选['1', '2', '3', '4', '5', '6', '7', '8', '9', '0']
	industry	string	可选	行业类型编码
	offset	string	可选	请求起始位置, 默认 0
	limit	string	可选	每次请求记录数, 默认 10
请求体 参数	无			
响应参数	code	string	调用失败时, 返回的错误码	
	msg	string	调用失败时, 返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时, 返回业务数据	

	data.list	array	产品信息集合, 如下的l表示 list 数组的单个对象标识
	l. total	number	设备数量
	l. desc	string	描述
	l. protocol	number	协议 1-泛协议 2-MQTT 3-CoAP
	l. created_time	string	创建时间
	l. product_id	string	产品 ID
	l. network	string	联网模式 1-其他 2-蜂窝 3-wifi 4-以太网
	l. ind_title	string	产品行业名称
	l. prod_ind	string	产品行业编码
	l. uid	string	产品用户 ID
	l. sec_key	string	产品权限 KEY
	l. name	string	产品名称
	l. manufacturer	string	厂商名称
	l.template_id	string	模板 id
	l.prod_chain	array	产品行业编码层级关系
	l. category	array	分类 ID
	l.category_name	string	分类名称
	data.meta	object	分页信息
	data.meta.limit	number	每次请求记录数
	data.meta.offset	number	请求记录起始位置
	data.meta.total	number	条数
请求示例	GET /common?action=SAProductList&version=1&product_id=wA10WBynvt		
响应示例	{		

例

```
"data": {
  "list": [
    {
      "total": 2,
      "protocol": 4,
      "created_time": "2021-08-31T08:00:12.846Z",
      "product_id": "wA10WBynvt",
      "network": "3",
      "category": ["66006c672de43d02590b098b"],
      "ind_title": "智慧城市",
      "prod_ind": "1",
      "prod_chain": ["3", "9"],
      "uid": 5,
      "template_id": "6687c206537561748c69f470",
      "sec_key": "RJKrcCGT22DYGSQ4KVXslu/Jnh3bezHzvhqCTFMk05Q=",
      "desc": "",
      "name": "qwe123",
      "manufacturer": "S1234567890",
      "category_name": "智能后视镜"
    }
  ],
  "meta": {
    "total": 1,
    "limit": 10,
    "offset": 0
  }
},
"requestId": "a25087f46df04b69b29e90ef0acfd115",
"success": true
}
```

1.4.8.3. 设备列表

方法	GET			
路径 URI	/common?action=SADeviceList&version=1			
请求头				
URL 请 求参数	name	string	可选	设备名称
	product_id	string	可选	产品 ID
	status	string	可选	设备状态, 可选['1'未激活, '2'在线, '3'离线]
	offset	string	可选	请求起始位置, 默认 0
	limit	string	可选	每次请求记录数, 默认 10
请求体 参数	无			
响应参 数	code	string		调用失败时, 返回的错误码
	msg	string		调用失败时, 返回的错误信息
	requestId	string		调用 API 时生成的请求标识
	success	boolean		接口是否调用成功
	data	—		调用成功时, 返回业务数据
	data.list	array		产品信息集合, 如下的 l 表示 list 数组的单个对象标识
	l. product_id	string		产品 ID
	l. name	string		设备名称
	l. node_type	number		节点类型 1: 直连设备, 2: 网关设备, 3: 网关子设备 (系统接入产品、设备没有该属性, 适用于本文档所有的节点类型字段)
	l. status	number		设备状态 1: 未激活, 2: 在线, 3:

			离线 【默认为未激活】
	l. last_time	string	设备最后一次在线时间
	l. created_time	string	创建时间
	l. from	string	设备来源(1:自主创建, 2:他人转移)
	l. lon	string	经度
	l. lat	string	纬度
	l. idid	string	设备 ID
	l. scene	array	场景 id
	l. product_name	string	产品名称
	data.meta	object	分页信息
	data.meta.limit	number	每次请求记录数
	data.meta.offset	number	请求记录起始位置
	data.meta.total	number	条数
请求示例	GET /common?action=SADeviceList&version=1		
响应示例	<pre>{ "data": { "list": [{ "name": "400A002090011001", "node_type": 1, "status": 3, "last_time": "2021-12-10T10:19:33.327Z", "from": 1, "lon": "" }] } }</pre>		

	<pre> "lat": "", "idid": "10015446", "product_id": "7ubgKilvbm", "created_time": "2021-11-30T08:22:53.519Z", "product_name": "水电费", "scene": ["64d205c4e51b4a6e3c125c4a"], }], "meta": { "total": 1, "limit": 10, "offset": 0 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true } </pre>
--	--

1.4.8.4. 产品中的设备数量统计

方法	GET			
路径 URI	/common?action=SAPProductDeviceStatistics&version=1			
请求头				
URL 请求参数	product_id	string	必填	产品 id
请求体参数	无			
响应参数	code	string	调用失败时，返回的错误码	

	msg	string	调用失败时，返回的错误信息
	requestId	string	调用 API 时生成的请求标识
	success	boolean	接口是否调用成功
	data	—	调用成功时，返回的业务数据
	data.unactive	number	未激活设备数
	data.online	number	在线设备数
	data.offline	number	离线设备数
	data.total	number	总的设备数
请求示例	GET /common?action=SAProductDeviceStatistics&version=1&product_id=7ubgKilvhm		
响应示例	<pre>{ "data": { "unactive":6, // 未激活数 "online":0, // 在线数 "offline":8, // 离线数 "total":14 // 总数 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.8.5. 设备详情

方法	GET
路径 URI	/common?action=SADeviceDetail&version=1&product_id=lsibd9

	&device_name=no001			
请求头				
URL 请求参数	product_id	string	必填	产品 id
	device_name	string	必填	设备名称
请求体参数	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回的业务数据	
	data.device_name	string	设备名称	
	data.product_id	string	产品 ID	
	data.product_name	string	产品名称	
	data.desc	string	设备描述	
	data.status	int	设备状态 1-未激活 2-在线 3-离线	
	data.node_type	int	节点类型 1-直连设备	
	data.protocol	int	协议类型 1-泛协议 2-MQTT 3-CoAP	
	data.ip	string	设备连接 ip	
	data.create_time	string	创建时间	
	data.last_time	string	最后一次在线时间	
	data.active_time	string	激活时间	
	data.scene	array	场景 id	

	data.sec_key	string	设备密钥
请求示例	GET /common?action=SADeviceDetail&version=1&product_id=lsibd9 &device_name=no001		
响应示例	<pre>{ "data": { "device_name": "no001", "product_id": "9MaNe52pNO", "product_name": "空气净化器", "active_time": "2020-06-19T08:11:27.801Z", "create_time": "2020-06-19T06:09:22.550Z", "desc": "设备 1", "ip": "192.168.200.139", "last_time": "2020-06-19T09:48:15.027Z", "node_type": 1, "protocol": 2, "sec_key": "UQd3K9lXR/EbLXeJc50lJfvvkTVdu5uFgbfz48/fl5k=", "status": 3, "scene": ["64d205c4e51b4a6e3c125c4a"], }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }</pre>		

1.4.8.6. 设备状态查询

方法	GET
路径 URI	/common?action=SADeviceStatus&version=1&product_id=9MaNe52pNO &device_name=no001

请求头				
URL 参数	product_id	string	必填	产品 ID
	device_name	string	必填	设备名称
请求体	无			
响应参数	code	string	调用失败时，返回的错误码	
	msg	string	调用失败时，返回的错误信息	
	requestId	string	调用 API 时生成的请求标识	
	success	boolean	接口是否调用成功	
	data	—	调用成功时，返回业务数据	
	data.status	int	设备状态 1-未激活 2-在线 3-离线	
请求示例	GET /common?action=SADeviceStatus&version=1&product_id=9MaNe52pNO &device_name=no001			
响应示例	{ "data": { status: 1 }, "requestId": "a25087f46df04b69b29e90ef0acfd115", "success": true }			

1.4.9. LwM2M 即时命令

1.4.9.1. 即时命令-设备资源列表获取

方法	GET
路径 URI	/lwm2m-online?action=resources&version=1
请求头	authorization - 鉴权信息

URL 参数	imei	string	必填	nbiot 设备的身份码
	obj_id	int	可选	设备的 objectid, 根据终端设备 sdk 确定
请求体	无			
响应参数	code	int		调用错误码, 为 0 表示调用成功
	msg	string		错误描述, 为"succ"表示调用成功
	data	—		调用成功时, 返回业务数据
	data.status	int		设备状态 1-未激活 2-在线 3-离线
	data.total_count	int		返回的条目数
	data.item	array-json		返回的条目详情, 见 item 描述表
	data.item.obj_id	int		设备的 object id, 根据终端设备 sdk 确定
	data.item.instances	array-json		obj_id 对象下的实例条目, 见 instances 描述表
	data.item.instances.inst_id	int		设备的 instances id, 根据终端设备 sdk 确定
	data.item.instances.resources	array-int		设备 instances id 下所有的资源列表
请求示例	GET /lwm2m-online?action=resources&imei=***&obj_id=3200			
响应示例	<pre>{ "code": 0, "msg": "succ", "data": { "total_count": 123, "item": [</pre>			

	<pre> { "obj_id":3200, "instances":[{ "inst_id":0, "resources":[5500,5050] }, {...}, ...] }, {.....},] } } </pre>
--	--

1.4.9.2. 即时命令-资源发现

方法	GET			
路径 URI	/lwm2m-online?action=discover&version=1			
请求头	authorization - 鉴权信息			
URL 参 数	imei	string	是	nbiot 设备的身份码，必填
	obj_id	int	是	nbiot 设备的 object id，对应到平台模型中为数据流 id，必填
	obj_inst_id	int	否	设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，选填
	timeout	int	否	超时时间,可选,默认 25 秒，取值

				范围[5,40]
请求体	无			
响应参数	code	int	调用错误码，为 0 表示调用成功	
	msg	string	错误描述，为"succ"表示调用成功	
请求示例	GET /lwm2m-online?action=discover&imei=***&obj_id=3200			
响应示例	<pre>{ "code": 0, "msg": "succ", "data": [{ "obj_inst_id": 3, "res_id": [3, 2, 4] }, {}, ] }</pre>			

1.4.9.3. 即时命令-资源订阅

方法	POST			
路径 URI	/lwm2m-online?action=observe&version=1			
请求头	authorization - 鉴权信息			
URL 参数	无			
请求体	imei	string	是	nbiot 设备的身份码，必填

	cancel	bool	是	true 为取消订阅，false 为订阅，必填
	obj_id	int	是	nbiot 设备的 object id，对应到平台模型中为数据流 id，必填
	obj_inst_id	int	否	设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，选填
	res_id	int	否	nbiot 设备的资源 id，选填
	pmin	int	否	上传数据的最小时间间隔，默认为 0，此时有数据就上传
	pmax	int	否	上传数据的最大时间间隔
	gt	double	否	当数据值大于该值时上传
	lt	double	否	当数据值小于该值时上传
	st	double	否	当前后两个数据点值相差大于或者等于该值时
	timeout	int	否	请求超时时间，默认 25(单位：秒)，取值范围[5,40]
响应参数	code	int		调用错误码，为 0 表示调用成功
	msg	string		错误描述，为"succ"表示调用成功
请求示例	POST /lwm2m-online?action=observe { "imei":xxxx, // nbiot 设备的身份码，由 15 位数字组成； "cancel":true false, //true 为取消订阅，false 为订阅，必填 "obj_id":xxxx, // nbiot 设备的 object id，对应到平台模型中为数据流 id，必填 "obj_inst_id": xxxx, //设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，选填 "res_id": 123, // nbiot 设备的资源 id，选填			

	"pmin":11, //上传数据的最小时间间隔 int 类型，默认为 0，此时有数据就上传 "pmax":123, //上传数据的最大时间间隔， int 类型， 可选 "gt":12, //当数据大于该值上传， double 类型， 可选 "lt":233, // 当数据小于该值上传， double 类型， 可选 "st":12 //当两个数据点相差大于或者等于该值上传， double 类型， 可选 }
响应示例	<pre>{ "code": 0, "msg": "succ" }</pre>

1.4.9.4. 即时命令-读设备资源

方法	GET			
路径 URI	/lwm2m-online?action=read&version=1			
请求头	authorization - 鉴权信息			
URL 参数	imei	string	是	nbiot 设备的身份码
	obj_id	int	是	设备的 objectid， 对应到平台模型中为数据流 id
	obj_inst_id	int	否	nbiot 设备 object 下具体一个 instance 的 id ， 对应到平台模型中数据点 key 值的一部分， 选填
	res_id	int	否	nbiot 设备的资源 id， 选填
	timeout	int	否	超时时间,可选,默认 25 秒， 取值范围[5,40]

请求体	无		
响应参数	code	int	调用错误码，为 0 表示调用成功
	msg	string	错误描述，为"succ"表示调用成功
	data	array-json	接口调用成功之后返回的设备资源相关信息，见 data 描述表
	data.obj_inst_id	int	对象实例 id
	data.res	array-json	res 列表
	data.res.res_id	int	资源 id
	data.res.val	object	读取资源的值，可以是 bool,string,long,double 等类型
请求示例	GET /lwm2m-online?action=read&imei=**&obj_id=***		
响应示例	<pre>{ "code": 0, "msg": "succ", "data": [{ "obj_inst_id":123, "res":[{ "res_id":1234, "val": Object //可为 boolean、string、long、double 类型数据 }, ] }], }</pre>		

	]
	}

1.4.9.5. 即时命令-写设备资源

方法	POST			
路径 URI	/lwm2m-online?action=write&version=1			
请求头	authorization - 鉴权信息			
URL 参 数	imei	string	是	nbiot 设备的身份码
	obj_id	int	是	设备的 object id，对应到平台模型中为数据流 id，必填
	obj_inst_id	int	是	nbiot 设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，必填
	mode	int	是	write 的模式，只能是 1 或者 2
	timeout	int	否	请求超时时间，默认 25(单位：秒)，取值范围[5,40]
请求体	data	array-j son	是	写设备资源的 json 数组，大小限制 2k，见 data 请求参数描述表
	res_id	int	是	指定 write 操作的资源 id
	type	int	否	目前支持为 1 和 2： 1 代表该资源 type 为 Opaque，此时 val 字段为该二进制对应的十六进制字符串； 2 代表该资源 type 为 Time，此时 val 字段为时间戳（单位为秒，数值）；不写代表该资源 type 为基础数据类型
	val	object	是	根据指定资源的类型决定 val 的

				数值类型，可为 boolean、string、long、double
响应参数	code	int		调用错误码，为 0 表示调用成功
	msg	string		错误描述，为"succ"表示调用成功
请求示例	POST /lwm2m-online?action=write 非 opaque 类型： { "data":[{ "res_id":12, "val":121 }] } //HTTP 内容部分必须存在。 opaque 类型： { "data":[{ "res_id":12, "type":1, "val":121 }] } //HTTP 内容部分必须存在。			
响应示例	{ "code": 0, "msg": "succ"			

	}
--	---

1.4.9.6. 即时命令-命令下发

方法	POST			
路径 URI	/lwm2m-online?action=execute&version=1			
请求头	authorization - 鉴权信息			
URL 参 数	imei	string	是	nbiot 设备的身份码
	obj_id	int	是	nbiot 设备的 object id，对应到平台模型中为数据流 id，必填
	obj_inst_id	int	是	设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，必填
	res_id	int	是	nbiot 设备的资源 id，必填
	timeout	int	否	请求超时时间，默认 25(单位：秒)，取值范围[5,40]
请求体	args	string	是	命令字符串，大小限制 2k
响应参 数	code	int		调用错误码，为 0 表示调用成功
	msg	string		错误描述，为"succ"表示调用成功
请求示 例	POST /lwm2m-online?action=execute { "args": "ping" // 字符串 } // HTTP 内容部分选填。			
响应示 例	{ "code": 0, "msg": "succ" }			

1.4.10. LwM2M 缓存命令

1.4.10.1. 缓存命令-读设备资源

方法	GET			
路径 URI	/lwm2m-offline?action=read&version=1			
请求头	authorization - 鉴权信息			
URL 参 数	imei	string	是	nbiot 设备的身份码，必填
	obj_id	int	是	设备的 object id，对应到平台模型中为数据流 id，必填
	obj_inst_id	int	否	nbiot 设备 object 下具体一个 instance 的 id，对应到平台模型中数据点 key 值的一部分，选填
	res_id	int	否	nbiot 设备的资源 id，选填
	valid_time	string	否	命令开始生效时间，可选（不填时默认为 OneNET 当前时间），填写必须大于 OneNET 服务器的当前时间
	expired_time	string	是	命令过期时间，必须大于 valid_time
	retry	int	否	表示失败重试次数（等待下一次设备 update 或者上线），可选（不填时默认为 3），填写时必须在[0, 10]之间
	timeout	int	否	过期时间，填写时必须在[5,40]s 之间；不填写默认为 25s
	trigger_msg	int	否	命令触发的上行消息类型，填写时必须在[1,7]之间；不填写默认为 7

	bin2hex	boolean	否	truefalse, 默认 false, 如果 bin2hex 为 true, 资源类型为 Opaque 的值将被转换成十六进制字符串返回
请求体	无			
响应参数	code	int	调用错误码, 为 0 表示调用成功	
	msg	string	错误描述, 为"succ"表示调用成功	
	data	json	接口调用成功之后返回的设备相关信息, 见 data 描述表	
	uuid	string	缓存命令 uuid	
请求示例	GET /lwm2m-offline?action=read&imei=86582003***&valid_time=2016-08-05T08:00:00&expired_time=2016-08-06T08:00:00&obj_id=1212&obj_inst_id=1212&res_id=123 请求参数示例: trigger_msg 触发类型: 1. REGISTER 2. UPDATE 3. REGISTER UPDATE 组合 4. NOTIFY 5. REGISTER NOTIFY 组合 6. UPDATE NOTIFY 组合 7. NOTIFY UPDATE REGISTER 组合 { "imei":121, // nbiot 设备的身份码, 必填 "obj_id":1212, // 设备的 object id, 对应到平台模型中为数据流 id, 必填 "obj_inst_id": 1212, // nbiot 设备 object 下具体一个 instance 的 id , 对应到平台模型中数据点 key 值的一部分, 选填			

	<pre> "res_id": 2122, // nbIoT 设备的资源 id, 选填 "valid_time": "2018-03-08T17:30:00", // 命令开始生效时间, 可选 (不填时默认为 OneNET 当前时间), 填写必须大于 OneNET 服务器的当前时间 "expired_time": "2018-03-09T17:30:00", // 命令过期时间, 必填且大于 valid_time "retry": 3, // 表示失败重试次数 (等待下一次设备 update 或者上线), 可选 (不填时默认为 3), 填写时必须要在 [3, 10] 之间 "bin2hex": true // 可选, 取值为 true/false, 默认 false, 如果 bin2hex 为 true, 资源类型为 Opaque 的值将被转换成十六进制字符串返回 } </pre>
响应示例	<pre> { "code": 0, "msg": "succ", "data": { "uuid": "42742677-adc3-54ca-83a1-5aaaf71482f8" // 缓存命令 uuid } } </pre>

1.4.10.2. 缓存命令-写设备资源

方法	POST			
路径 URI	/lwm2m-offline?action=write&version=1			
请求头	authorization - 鉴权信息			
URL 参数	imei	string	是	nbIoT 设备的身份码, 必填
	obj_id	int	是	设备的 object id, 对应到平台模型中为数据流 id, 必填

	obj_inst_id	int	是	nbiot 设备 object 下具体一个 instance 的 id ， 对应到平台模型中数据点 key 值的一部分， 必填
	mode	int	是	write 的写模式， 只能是 1 或者 2
	valid_time	string	否	命令开始生效时间， 可选（不填时默认为 OneNET 当前时间）， 填写必须大于 OneNET 服务器的当前时间
	expired_time	string	是	命令过期时间戳， 必填且大于 valid_time
	retry	int	否	[0-10]之间， 表示失败重试次数（等待下一次设备 update 或者上线）， 可选（不填时默认为 3）， 填写时必须在[0, 10]之间
	timeout	int	否	过期时间， 填写时必须在[5,40]s 之间； 不填写默认为 25s
	trigger_msg	int	否	命令触发的上行消息类型， 填写时必须在[1,7]之间； 不填写默认为 7
请求体	data	array-json	是	写设备资源的 json 数组， 大小限制 2k， 见 data 请求参数描述表
	res_id	int	是	nbiot 设备的资源 id
	type	int	否	目前支持为 1 和 2， 1 代表该资源 type 为 Opaque， 此时 val 字段为该二进制对应的十六进制字符串； 2 代表该资源 type 为 Time， 此时 val 字段为时间戳（单位为秒， 数值）； 不写代表该资源 type 为基础数据类型
	val	object	是	值， 可为 boolean、string、long、

				double
响应参数	code	int	调用错误码，为 0 表示调用成功	
	msg	string	错误描述，为"succ"表示调用成功	
	data	json	接口调用成功之后返回的设备相关信息，见 data 描述表	
	uuid	string	缓存命令 uuid	
请求示例	POST /lwm2m-offline?action=write trigger_msg 触发类型： 1. REGISTER 2. UPDATE 3. REGISTER UPDATE 组合 4. NOTIFY 5. REGISTER NOTIFY 组合 6. UPDATE NOTIFY 组合 7. NOTIFY UPDATE REGISTER 组合 非 opaque 类型： { "data": [{ "res_id":12, "val":121 }] } opaque 类型： { "data": [{			

	<pre> "res_id":12, "type":1, "val":"121A" }] } </pre>
响应示例	<pre> { "code": 0, "msg": "succ", "data": { "uuid":"42742677-adc3-54ca-83a1-5aaaf71482f8"//缓存命令 uuid } } </pre>

1.4.10.3.缓存命令-命令下发

方法	POST			
路径 URI	/lwm2m-offline?action=execute&version=1			
请求头	authorization - 鉴权信息			
URL 参数	imei	string	是	设备模组 IMEI, 必填
	obj_id	int	是	nbiot 设备的 object id , 对应到平台模型中为数据流 id, 必填
	obj_inst_id	int	是	设备 object 下具体一个 instance 的 id , 对应到平台模型中数据点 key 值的一部分, 必填
	res_id	int	是	nbiot 设备的资源 id, 必填

	valid_time	string	否	命令开始生效时间戳，可选（不填时默认为 OneNET 当前时间），填写必须大于 OneNET 服务器的当前时间
	expired_time	string	是	命令过期时间戳，必填且大于 valid_time
	retry	int	否	[0-10]之间，表示失败重试次数（等待下一次设备 update 或者上线），可选（不填时默认为 3），填写时必须要在[0, 10]之间
	timeout	int	否	过期时间，填写时必须要在[5,40]s之间；不填写默认为 25s
	trigger_msg	int	否	命令触发的上行消息类型，填写时必须要在[1,7]之间；不填写默认为 7
请求体	args	string	否	命令字符串，大小限制 2k
响应参数	code	int		调用错误码，为 0 表示调用成功
	msg	string		错误描述，为"succ"表示调用成功
	data	json		接口调用成功之后返回的设备相关信息，见 data 描述表
	uuid	string		缓存命令 uuid
请求示例	POST /lwm2m-offline?action=execute&imei=86582003***&valid_time=2016-08-05T08:00:00&expired_time=2016-08-06T08:00:00&obj_id=1212&obj_inst_id=1212&res_id=123 trigger_msg 触发类型： 1. REGISTER 2. UPDATE 3. REGISTER UPDATE 组合			

	4. NOTIFY 5. REGISTER NOTIFY 组合 6. UPDATE NOTIFY 组合 7. NOTIFY UPDATE REGISTER 组合 POST data <pre>{ "args": "ping" //字符串 } //HTTP 内容部分选填</pre>
响应示例	<pre>{ "code": 0, "msg": "succ", "data": { "uuid": "42742677-adc3-54ca-83a1-5aaaf71482f8" //缓存 } "命令 uuid" }</pre>